

1. Respuesta:

- I. A. ==
- II. D. !
- III. G. Cast with (boolean)

Explicación:

Los operadores == y ! pueden utilizarse con variables de tipo boolean: == permite comparar valores booleanos y ! permite invertir su valor. Finalmente, (boolean) puede utilizarse para realizar una conversión explícita a este tipo. Por lo tanto, las opciones A, D y G son correctas.

2. Respuesta:

- I. A. int
- II. B. long
- III. D. double

Explicación:

Aunque apples es byte y oranges es short, al realizar la suma ambos valores se promocionan automáticamente a int. Por lo tanto, el resultado de apples + oranges es de tipo int y puede asignarse a tipos más grandes como long y double. En cambio, no puede asignarse directamente a short o byte. Por lo tanto, las opciones A, B y D son correctas.

3. Respuesta:

- I. B. Cast ear on line 4 to int.
- II. C. Change the data type of ear on line 3 to short.
- III. D. Cast 2 * ear on line 4 to int.
- IV. F. Change the data type of hearing on line 4 to long

Explicación:

La variable ear es de tipo long, por lo que la expresión 2 * ear también produce un resultado de tipo long. Ese valor no puede asignarse directamente a una variable int sin una conversión explícita. Por eso, se puede convertir ear o toda la expresión a int, cambiar ear a short para que la operación se promocione a int, o cambiar hearing a long. Por lo tanto, las opciones B, C, D y F son correctas.

4. Respuesta:

- I. B. true, 20, false

Explicación

En la expresión (teeth != 10) ^ (wolf = false), teeth != 10 es true porque teeth vale 20. Además, wolf = false asigna false a wolf y devuelve ese valor. El operador ^ realiza un XOR: como los valores son true y false, el resultado es true, por lo que canine queda en true. teeth no cambia y wolf queda en false. Por lo tanto, la salida es true, 20, false y la opción B es correcta.

5. Respuesta:

I. A. +, *, %, - -

II. C. =, ==, !

Explicación:

Para que una opción sea correcta, los operadores deben aparecer en orden creciente o con la misma precedencia.

Las opciones A y C cumplen esta condición. En A, los operadores están ordenados de menor a mayor precedencia. En C, = tiene menor precedencia que ==, y == tiene menor precedencia que !. Por lo tanto, las opciones A y C son correctas.

6. Respuesta:

I. F. None of the above.

Explicación:

En la línea 3, (int)fruit convierte fruit a int, pero al sumarlo con vegetables, que es float, el resultado de la expresión es float. Ese valor no puede devolverse directamente como long sin realizar una conversión explícita.

Por lo tanto, el código no compila debido a la línea 3. Como ninguna de las opciones indica este error, la respuesta correcta es F. None of the above.

7. Respuesta:

I. D. true- false- false

Explicación:

Primero, clear es true porque vis > 1 es verdadero y (vis < 9 || ph < 2) también es verdadero. Después, safe es false porque vis > 2 es falso. Al utilizar &&, la segunda expresión no se evalúa, por lo que ph++ no se ejecuta y ph permanece en 7. Finalmente, tasty es false porque 7 <= --ph equivale a 7 <= 6, ya que --ph disminuye ph de 7 a 6. Por lo tanto, la salida es true- false- false y la opción D es correcta.

8. Respuesta:

I. A. 4 - 1

Explicación:

En la línea 5, pig++ incrementa temporalmente el valor de pig, pero la expresión devuelve primero el valor original (4), que luego se vuelve a asignar a pig. Por eso, pig permanece en 4. En la línea 7, goat -= 1.0 es válido porque 1.0 es un double, pero el operador de asignación compuesta realiza automáticamente la conversión necesaria al tipo de goat. Por lo tanto, goat pasa de 2 a 1. La salida es 4 - 1, por lo que la opción A es correcta.

9. Respuesta:

I. A. 1

II. D. 4

III. E. 5

Explicación:

Primero, $a > 2$ es falso, por lo que se ejecuta $b++$. Se imprime 4 y luego b aumenta a 5. Después, $a != c$ es falso porque ambos valen 2. Por lo tanto, se ejecuta $b++$, que devuelve 5 y después incrementa b a 6. La asignación $b = 5$ deja b nuevamente en 5, por lo que se imprime 5. Finalmente, $a > b$ es falso ($2 > 5$), así que se ejecuta la última opción del operador ternario y se imprime 1. Los valores únicos que se imprimen son 1, 4 y 5, por lo que las opciones A, D y E son correctas.

10. Respuesta:

I. G. The code does not compile.

Explicación:

El código no compila porque en la línea de zebra, aunque `weight` y `height` se convierten a `byte`, al realizar la multiplicación ambos valores se promocionan a `int`. Por lo tanto, el resultado de `(byte) weight * (byte) height` es de tipo `int` y no puede asignarse directamente a una variable `short` sin un `cast` explícito.

Por lo tanto, la opción G. The code does not compile es correcta.

11. Respuesta:

I. D. 2, 0, 5

Explicación:

En la línea 11, primero se realiza $2 * 4 = 8$ y después $8 \% 3 = 2$.

En la línea 12, $3 * 2 = 6$ y $6 \% 3 = 0$.

En la línea 13, $1 \% 2 = 1$ y luego $5 * 1 = 5$.

Por lo tanto, la salida es 2, 0, 5, por lo que la opción D es correcta.

12. Respuesta:

I. D. post- increment, pre- decrement

Explicación:

El operador post-incremento ($x++$) aumenta el valor de una variable, pero devuelve su valor original antes de incrementarlo. En cambio, el operador pre-decremento ($--x$) disminuye el valor primero y después devuelve el nuevo valor. Por lo tanto, la opción D. post-increment, pre-decrement es correcta.

13. Respuesta:

I. F. true- true- false

Explicación:

Primero, `goingToTheStore` es `true`: `sunny & raining` da `false`, y `false ^ sunday` da `true`.

Después, `goingToTheZoo` es `true` porque `sunday` es `true` y `raining` es `false`, por lo que `!raining` es `true`.

Finalmente, `stayingHome` es `false` porque `goingToTheStore && goingToTheZoo` es `true`, y al aplicar `!` el resultado es `false`.

Por lo tanto, la salida es true- true- false, por lo que la opción F es correcta.

14. Respuesta:

- I. B. The inequality operator (!=) can be used to compare objects.
- II. E. The return value of an assignment operation expression is the value of the newly assigned variable.
- III. G. The logical complement operator (!) cannot be used to flip numeric values.

Explicación:

El operador != puede utilizarse para comparar referencias de objetos y determinar si apuntan a objetos diferentes. Una expresión de asignación devuelve el valor que acaba de ser asignado a la variable, por lo que la opción E es correcta. Además, el operador ! solo puede utilizarse con valores boolean, por lo que no puede utilizarse para invertir valores numéricos. Por lo tanto, las opciones B, E y G son correctas.

15. Respuesta:

- I. D. ? :

Explicación:

El único operador de los mostrados que utiliza tres operandos es el operador ternario ? :. Este recibe una condición y dos posibles valores: uno si la condición es true y otro si es false. Los demás operadores de la lista son unarios o binarios. Por lo tanto, la opción D. ? : es correcta.

16. Respuesta:

- I. B. 1

Explicación:

Solo hay un error de compilación. En la primera línea, `1 * 2 + (long)3` produce un valor de tipo long, que no puede asignarse directamente a una variable int sin realizar un cast. Las demás líneas son válidas porque las conversiones explícitas permiten asignar los valores a los tipos correspondientes. Por lo tanto, hay 1 error de compilación y la opción B es correcta.

17. Respuesta:

- I. C. ticketsSold is 6.
- II. F. ticketsTaken is 4.

Explicación:

Primero, ticketsTaken vale 1 y ticketsSold vale 3. En `ticketsSold += 1 + ticketsTaken++`, el `ticketsTaken++` utiliza primero el valor 1 y después lo incrementa a 2. Por lo tanto, se suma $1 + 1 = 2$ a ticketsSold, que pasa a valer 5.

Después, `ticketsTaken *= 2` duplica su valor: $2 \times 2 = 4$. Finalmente, `ticketsSold += (long)1` suma 1, por lo que ticketsSold queda en 6.

Por lo tanto, las opciones C y F son correctas.

18. Respuesta:

I. C. ()

Explicación:

Los paréntesis () pueden utilizarse para cambiar el orden en que se evalúan las operaciones dentro de una expresión. Las demás opciones no se utilizan para modificar la precedencia de los operadores en Java. Por lo tanto, la opción C. () es correcta.

19. Respuesta:

I. B. start is - 128.

II. F. end is 12.

Explicación:

Primero, ++start incrementa start de 7 a 8. Por lo tanto, end += 8 hace que end pase de 4 a 12. Después, Byte.MAX_VALUE es 127, así que Byte.MAX_VALUE + 1 da 128. Al convertir 128 a byte, se produce un desbordamiento y el valor resultante es -128. Por lo tanto, start termina en -128 y end en 12.

Las opciones B y F son correctas.

20. Respuesta:

I. A. Unary operators are always executed before any surrounding numeric binary or ternary operators.

II. D. The post-decrement operator (- -) returns the value of the variable before the decrement is applied.

III. E. The ! operator cannot be used on numeric values

Explicación:

Los operadores unarios tienen mayor precedencia que los operadores binarios y ternarios que los rodean, por lo que se evalúan primero. El operador post-decremento (x--) devuelve el valor original de la variable antes de disminuirlo. Además, el operador ! solo puede utilizarse con valores boolean, por lo que no puede aplicarse a valores numéricos. Por lo tanto, las opciones A, D y E son correctas.

21. Respuesta:

I. E. - 9,- 8,9

Explicación:

~myFavoriteNumber realiza el complemento bit a bit de 8, dando como resultado -9. Por otro lado, -myFavoriteNumber cambia el signo de 8, por lo que plane vale -8.

Como bird y plane son diferentes, la condición del operador ternario es falsa y superman toma el valor 10. Finalmente, --superman lo reduce a 9, por lo que se imprime -9,-8,9.

Por lo tanto, la opción E es correcta.